

Dhaka Tesla Pool

Master Project Instructions — Version 2
(Complete Build Guide + Implementation Directive, merged)

Internship Take-Home Challenge | RoBenDevs
Deadline: September 27

This is the single authoritative document for building this project. It supersedes and merges the earlier 14-page instructions with the follow-up Implementation Directive. Where a requirement was strengthened by the directive, it is marked below. This document is the source of truth Claude Code should be pointed at from the start of the build.

Contents

- 1. Authority & Operating Principle
- 2. Project Understanding & Story Cast
- 3. Scope — What To Build, What Not To Build
- 4. System Architecture
- 5. Database Design (ERD)
- 6. Database Invariants & Constraints
- 7. Fare Model
- 8. Concurrency — Seat Claims AND Pool Creation
- 9. Transactions & Atomicity
- 10. State Machine & Transition Policy
- 11. Cancellation Rules
- 12. Authorization
- 13. Passwords & Authentication
- 14. Service-Layer Organization
- 15. Validation
- 16. Auditability
- 17. Technology Stack & Justifications
- 18. Git Workflow & Commit Rules
- 19. Testing Requirements (Expanded)
- 20. Frontend Requirements
- 21. AI Usage Policy
- 22. Documentation Synchronization
- 23. Do Not Add — Unless Explicitly Required
- 24. Claude Code Operating Protocol
- 25. Build Roadmap (Day-by-Day)
- 26. README & Video Requirements
- 27. Requirement-to-Evidence Completion Audit
- 28. Definition of Done
- 29. Final Submission Checklist
- 30. Hard Rules — Never Do These

1. Authority & Operating Principle

- This document is authoritative. Do not reinterpret, replace, or silently relax any explicit requirement in it.
- Where the specification is explicit, follow it exactly.
- Where something is genuinely unspecified, make the smallest defensible assumption, document it, implement it consistently, and ensure the README, tests, ERD, architecture diagram, and code all agree with each other.
- Do not add complexity merely to make the architecture look advanced.

The objective is a correct, explainable, production-minded MVP — not the largest or most sophisticated system.

2. Project Understanding & Story Cast

A ride-pooling MVP set in Dhaka with three actors: a **Passenger** requesting rides, a **Driver** running a fixed-capacity vehicle (a 'Tesla'), and a **Ride/Pool** that may combine multiple passengers into one trip. Evaluation covers correctness, process (git discipline), and the ability to explain every decision live.

Name	Role	Detail
Jashim	Driver	Owns the Tesla
Bullet	Vehicle	3-seat capacity, battery-powered
Nusrat	Passenger	Banani → Mohakhali
Rafiq	Passenger	Banani → Gulshan 1 (books ~2 min after Nusrat)
Shirin	Passenger	Books ~30s later — the last-seat concurrency case

Use this cast everywhere: seed data, tests, README examples, screenshots/demo flows, and the video. Never use generic placeholders like user1/driver1.

3. Scope — What To Build, What Not To Build

Build:

- Passenger: sign up/in, request ride (pickup, destination, seats), see estimated fare, track status, view history, cancel while valid.
- Driver/Tesla: sign in, go online/offline, own a Tesla with fixed capacity, see relevant requests, accept a ride/pool, mark arrival/start/complete, see passengers/seats and ride history.
- Pool/Ride Split: multiple requests may share one Tesla, occupied seats never exceed capacity, each passenger gets an individual fare, clear lifecycle, obvious pool membership.

Ride lifecycle (must match exactly):

REQUESTED -> MATCHED/ACCEPTED -> DRIVER_ARRIVED -> STARTED -> COMPLETED (+ CANCELLED)

4. System Architecture

Keep this three-layer architecture exactly:

```
Browser
|
v
Next.js (App Router) / React frontend
|   REST over HTTPS
v
Node.js API (Express)
|
v
```

Do NOT introduce microservices, Kafka, Kubernetes, Redis, queues, or similar infrastructure unless this document explicitly requires it. It currently does not.

Why REST, not GraphQL:

The resource model is shallow and known up front. REST is faster to implement, test, and document under this timeline.

Why Postgres:

Relational integrity and transactional row-locking are the actual mechanism used to solve the concurrency requirements in Sections 8–9.

5. Database Design (ERD)

Follow this schema exactly. Do not redesign it unless a genuine implementation conflict is found — if a detail must differ, update this document in the same change.

users

Column	Type	Notes
id	uuid PK	
name / phone	string	
password_hash	string	never plaintext — see Section 13
role	enum	passenger driver
created_at	timestamp	

teslas

Column	Type	Notes
id	uuid PK	
driver_id	uuid FK -> users	
label	string	e.g. 'Bullet'
capacity	int	must be positive — CHECK constraint
status	enum	offline online

zones / corridors

Column	Type	Notes
zones.id / name / corridor_id / lat / lng	text	predefined Dhaka areas, no live routing
corridors.id / name	—	e.g. 'Banani-Gulshan-Mohakhali'

ride_requests

Column	Type	Notes
id	uuid PK	
passenger_id	uuid FK -> users	
pickup_zone_id / destination_zone_id	uuid FK -> zones	must reference existing zones
seats_requested	int	must be positive — CHECK constraint
status	enum	requested matched driver_arrived started completed cancelled
pool_id	uuid FK -> pools	nullable
requested_at ... cancelled_at	timestamp	one per lifecycle stage

pools

Column	Type	Notes
id	uuid PK	
tesla_id	uuid FK -> teslas	
seats_occupied	int	0 <= seats_occupied <= tesla.capacity — CHECK
status	enum	forming active completed cancelled

pool_members

Column	Type	Notes
id	uuid PK	
pool_id	uuid FK -> pools	
ride_request_id	uuid FK, UNIQUE	a ride can only ever join one pool
seats / joined_at	int / timestamp	

pool_status_history / ride_status_history

Column	Type	Notes
id	uuid PK	
pool_id or ride_request_id	uuid FK	
event / from_status / to_status	enum	append-only, never edited or deleted
changed_at	timestamp	

fares

Column	Type	Notes
id	uuid PK	
ride_request_id	uuid FK -> ride_requests	
base_fare_paisa / distance_charge_paisa / pool_discount_paisa / total_fare_paisa	integer	
payment_method / payment_status	enum	cash teslapay_wallet / pending paid

6. Database Invariants & Constraints

STRENGTHENED IN v2 — new or expanded requirement from the implementation directive.

Data integrity must be enforced at the database level, not only in application code. Frontend or application-layer validation alone is not sufficient.

- **pool_members.ride_request_id** must be **UNIQUE** — a ride cannot belong to more than one pool.
- A Tesla may have at most one forming/active pool, via the partial unique index below.
- **0 <= pools.seats_occupied <= teslas.capacity** must always hold — enforce with a **CHECK** constraint.
- **seats_requested** must be positive — **CHECK** constraint.
- Tesla **capacity** must be positive — **CHECK** constraint.
- All foreign-key relationships must be enforced by the database, not assumed by application code.
- Use **NOT NULL**, **UNIQUE**, **CHECK**, and other database constraints wherever they directly enforce a specified invariant.

```
CREATE UNIQUE INDEX one_active_pool_per_tesla
  ON pools (tesla_id)
  WHERE status IN ('forming', 'active');

ALTER TABLE pools ADD CONSTRAINT seats_within_capacity
  CHECK (seats_occupied >= 0);

ALTER TABLE ride_requests ADD CONSTRAINT positive_seats
  CHECK (seats_requested > 0);

ALTER TABLE teslas ADD CONSTRAINT positive_capacity
  CHECK (capacity > 0);
```

Matching rule (deterministic, keep exactly):

A ride pools with another when they share the same pickup zone **AND** their destination zones belong to the same destination corridor. Do not replace this with route-overlap distance, live routing, maps, or GPS — the simple rule is deliberate: deterministic and hand-testable.

7. Fare Model

```
passengerFare = baseFare + distanceCharge - poolDiscount
```

- All money stored as **integer paisa** — never float or decimal.
- **baseFare** = 3000 paisa unless this document is deliberately updated.
- **perKmRate** = 1500 paisa/km unless this document is deliberately updated.
- **poolDiscount** = 20%, applied individually to each passenger's own subtotal.

The implementation must reproduce these exact, hand-checkable figures: **Nusrat = 7200 paisa, Rafiq = 6000 paisa**. Write an automated test asserting these exact values.

Passenger	Route	Distance	Subtotal	Discount (20%)	Total Fare
Nusrat	Banani → Mohakhali	4 km	9000 paisa	1800 paisa	7200 paisa
Rafiq	Banani → Gulshan 1	3 km	7500 paisa	1500 paisa	6000 paisa

8. Concurrency — Seat Claims AND Pool Creation

STRENGTHENED IN v2 — new or expanded requirement from the implementation directive.

Use PostgreSQL row-level locking with `SELECT . . . FOR UPDATE` inside a transaction for pool-capacity claims. The capacity check and the update must happen while the relevant row is locked.

Required conceptual transaction:

```
BEGIN
  Lock the pool row and read current occupancy/capacity.
  Check seats_occupied + requested_seats <= Tesla.capacity.
  Reject and roll back if the capacity check fails.
  Update seats_occupied.
  Insert pool membership.
  Insert the appropriate audit history.
```

New requirement beyond seat claims: also protect pool CREATION/matching against races. Do not assume a prior "does a compatible pool exist?" query is sufficient — two requests could both see 'no pool exists' and both try to create one. The database unique constraint (Section 6) is the final integrity protection. If a concurrent pool creation hits the unique constraint, handle the conflict safely: catch it and retry by finding the existing compatible pool rather than failing the request outright.

Required test: two simultaneous requests for the last seat must result in only one successful claim; concurrent pool creation must not produce two active/forming pools for the same Tesla.

9. Transactions & Atomicity

STRENGTHENED IN v2 — new or expanded requirement from the implementation directive.

Any operation that changes multiple pieces of related state must be atomic (single transaction, all-or-nothing). Pay particular attention to:

- **Joining a pool:** capacity check + membership row + occupancy update + history record.
- **Cancelling a pooled ride:** ride status + membership/occupancy effects + history record.
- **Any state transition:** current status update + history entry.
- Any operation where a partial update could leave the pool or ride in an inconsistent state.

10. State Machine & Transition Policy

STRENGTHENED IN v2 — new or expanded requirement from the implementation directive.

REQUESTED -> MATCHED/ACCEPTED -> DRIVER_ARRIVED -> STARTED -> COMPLETED (+ CANCELLED)

- Implement explicit transition validation — never allow arbitrary status updates from API clients.
- Create a **single authoritative transition policy** in the backend service layer (one place that decides what's allowed, not scattered checks).
- Explicitly reject invalid transitions, e.g.: STARTED → REQUESTED, COMPLETED → REQUESTED, CANCELLED → STARTED, COMPLETED → CANCELLED.
- Every successful transition must create an append-only ride_status_history record.

11. Cancellation Rules

STRENGTHENED IN v2 — new or expanded requirement from the implementation directive.

The specification requires cancellation rules but leaves some details to engineering judgment. Choose a simple, defensible rule and document it **before** implementing.

- Define exactly which ride states allow passenger cancellation.
- Reject cancellation after the ride has started (status = STARTED or later).
- Define whether and how pool membership is removed when cancellation is allowed.
- Update seats_occupied consistently when a passenger leaves a pool.
- Record the relevant ride and pool history for the cancellation.
- Keep all related updates atomic (see Section 9).
- Add tests for both permitted and rejected cancellation cases.

12. Authorization

STRENGTHENED IN v2 — new or expanded requirement from the implementation directive.

Authentication alone is not sufficient. Every protected resource must enforce ownership/role authorization.

- A passenger can view and modify only their own ride.
- A passenger cannot access another passenger's ride by guessing a UUID.
- A passenger cannot execute driver-only operations.
- A driver can operate only their own Tesla and authorized rides/pools.
- Driver state-changing operations must verify both role AND ownership/association — not role alone.
- Never use frontend button visibility as the authorization mechanism — enforce it server-side, every time.

13. Passwords & Authentication

STRENGTHENED IN v2 — new or expanded requirement from the implementation directive.

- Never store plaintext passwords — store password_hash only.
- Use a standard secure password hashing mechanism (e.g. bcrypt/argon2).
- JWT should identify the authenticated user and their role.
- Validate authentication at the backend for every protected route — not just on the frontend.

14. Service-Layer Organization

STRENGTHENED IN v2 — new or expanded requirement from the implementation directive.

Prefer a clear separation: route → controller → service → database/repository. Business rules should not be scattered across frontend code or large route handlers.

Service	Owns
Ride service	Ride lifecycle and cancellation rules
Pool service	Matching, membership, capacity, pool consistency

Service	Owns
Fare service	Fare calculation
Authorization middleware/service	Identity and role checks

Database transactions remain close to the business operation they protect (i.e. inside the service that owns that operation, not spread across layers).

15. Validation

STRENGTHENED IN v2 — new or expanded requirement from the implementation directive.

- Validate request bodies and route parameters.
- Validate UUIDs and enum values.
- Validate `seats_requested > 0`.
- Validate referenced zones exist.
- Reject malformed or impossible requests.

Validation does not replace authorization or database constraints — all three layers (validation, authorization, DB constraints) are required, not alternatives to each other.

16. Auditability

Keep `ride_status_history` and `pool_status_history` as append-only audit trails. Do not replace them with a generic event-sourcing infrastructure.

The system should be able to answer: what happened, when, to which ride/pool, and how occupancy changed.

17. Technology Stack & Justifications

Layer	Choice	Why
Frontend	Next.js (App Router)	File-based routing for passenger/driver flows
Backend	Node.js + Express	Simple, well-documented, minimal setup overhead
Database	PostgreSQL	Transactional locking needed for concurrency requirements
Auth	JWT, no refresh tokens	Simplest defensible option for an MVP
Testing	Jest	Standard for Node, no setup friction
Deployment	Free tier, Docker fallback	Per Section 3; no paid infra

README must explain realistic alternatives, why each selected technology fits this MVP specifically, and what would justify switching later.

18. Git Workflow & Commit Rules

- **master** — long-lived integration branch.
- **feature/*** — one branch per logical feature.
- **pre-release** — integration fixes, documentation, deployment checks.
- **release/v1.0.0** — final version shown in the video.
- Never push feature development directly to master.
- Never manufacture a fake history after the project is already complete.
- Use incremental commits as the feature is actually built.
- Commit format: `type(scope): short description` — types: feat, fix, refactor, test, docs, chore, build.
- Avoid meaningless micro-commits and avoid giant finished-system commits.

19. Testing Requirements (Expanded)

STRENGTHENED IN v2 — new or expanded requirement from the implementation directive.

Tests must cover behavior and invariants, not merely endpoint availability.

- Bullet capacity can never be exceeded.
- Two simultaneous requests for the last seat result in only one successful claim.
- Concurrent pool creation cannot produce multiple active/forming pools for the same Tesla.
- Invalid state transitions are rejected; all important valid state transitions work.
- Nusrat's fare equals exactly 7200 paisa; Rafiq's fare equals exactly 6000 paisa.
- A passenger cannot view or modify another passenger's ride.
- A passenger cannot execute driver-only operations.
- A driver cannot operate another driver's Tesla.
- Cancellation works only in the defined valid states; cancellation after the ride has started is rejected.
- Pool occupancy is updated correctly after an allowed cancellation.
- A ride cannot be added to two pools.
- History records are created for every state/occupancy change.
- Database constraints reject structurally invalid data (not just application-level checks).

20. Frontend Requirements

Correctness before polish.

- Implement passenger and driver flows clearly.
- Show loading, error, and empty states.
- Show the current ride lifecycle state.
- Show each passenger only their own fare/status.
- Show pool membership and seats clearly.
- Do not spend disproportionate time on animations or visual polish.
- Do not add maps or live routing.

21. AI Usage Policy

- AI use is allowed. Do not hide it.
- README must state which AI tools were used and what for.
- Include at least one accepted AI suggestion.
- Include at least one rejected or changed AI suggestion, and explain why.
- Do not ship code you cannot explain, debug, redesign, or modify.

22. Documentation Synchronization

Whenever implementation changes a documented architecture, schema, rule, or behavior, update the relevant documentation in the same logical change. The ERD, architecture diagram, README, tests, and implementation must all describe the same system at all times — never let them drift apart.

23. Do Not Add — Unless Explicitly Required

STRENGTHENED IN v2 — new or expanded requirement from the implementation directive.

- Real map/routing integration
- Real payment gateway
- Ratings/reviews
- Redis
- Kafka
- Kubernetes
- Microservices
- Message queues
- Complex event-sourcing infrastructure
- Live GPS tracking
- Chat
- Surge pricing
- Unnecessary notification systems
- Unnecessary analytics dashboards

24. Claude Code Operating Protocol

STRENGTHENED IN v2 — new or expanded requirement from the implementation directive.

This governs how Claude Code behaves for the entire duration of the build:

- Work as a careful engineering pair. Before each significant implementation step, state what requirement is being addressed, what is being changed, why, what invariant it protects, and how it will be tested.
- Do not silently skip requirements. Do not silently change the schema or architecture.
- If a requirement is ambiguous, stop and identify the ambiguity rather than inventing a hidden assumption.
- Maintain a running checklist throughout implementation. A feature is not complete merely because its happy path works.
- Flag every non-trivial design decision clearly, so it can be logged for the README's AI Usage section.
- Never mark a feature branch done without confirming it meets the Definition of Done (Section 28).

25. Build Roadmap (Day-by-Day)

Branch	What	When
feature/repo-scaffold	docker-compose.yml, .env.example, folder structure, health check	Day 1
feature/db-schema	Migrations incl. all constraints from Section 6, seed script with the cast	Day 1
feature/passenger-auth	Passenger sign up/in, password hashing, JWT	Day 1-2
feature/driver-auth	Driver sign up/in, online/offline toggle	Day 1-2
feature/ride-request	Create ride request, estimated fare, status tracking, validation	Day 2
feature/tesla-pooling	Matching, pool creation race protection, capacity lock, transition policy	Day 2
feature/driver-flow	Driver sees requests, accepts, marks arrived/started/completed	Day 2-3
feature/cancellation	Cancellation rules per Section 11, atomic updates + history	Day 2-3
feature/authorization	Ownership/role middleware across all protected routes	Day 2-3
feature/frontend-*	UI for all flows — can start in parallel once API contracts are stable	Day 2-3
pre-release	Integration fixes, docs, deploy checks, requirement audit begins	Day 3
release/v1.0.0	Final cut, deploy, record video, complete audit	Day 3

26. README & Video Requirements

README must cover, at minimum:

- Summary, problem statement, features implemented, screenshots/GIFs
- Architecture diagram and ERD/database diagram
- Tech stack, project structure, prerequisites
- Environment variables (.env.example, never real secrets)
- Local setup, Docker instructions, migration/seed instructions
- How to run frontend/backend and tests; demo credentials
- Deployment URL, API overview, key decisions/trade-offs, known limitations, next improvements
- AI Usage section and demo video link

Six-minute video structure:

Time	Content
0:00-1:00	Understanding of the problem, users, and core idea — in your own words
1:00-3:00	How it was engineered — architecture, backend, frontend, DB design, lifecycle, one key decision, one trade-off
3:00-6:00	Product tour — passenger flow, driver flow, pooling, fare/status, one edge case, deployment if available

27. Requirement-to-Evidence Completion Audit

STRENGTHENED IN v2 — new or expanded requirement from the implementation directive.

Before declaring the project complete, perform a requirement-by-requirement audit against this document.

For EVERY requirement, record:

- Requirement text or concise requirement identifier
- Implementation location (file/module/function)
- Test covering the requirement
- Documentation/README location
- Whether it was manually verified where appropriate
- Any limitation or deviation

If there is a deviation, explain why and update this document consistently. Suggested audit table format:

Requirement	Implemented in	Test	Docs	Verified	Deviation
e.g. Seat capacity never exceeded	pool.service.ts	pool.test.ts	README \$Design	Yes	None

28. Definition of Done

STRENGTHENED IN v2 — new or expanded requirement from the implementation directive.

- All required features implemented; required architecture followed
- Database schema and constraints match the specification exactly
- Concurrency behavior tested under the specified last-seat race AND concurrent pool creation

- Authorization tests pass; state transition tests pass; fare tests pass against documented examples
- Cancellation rules documented and tested; audit history working
- Docker Compose runs from a clean environment; .env.example complete, no secrets committed
- Migrations and seed data work; README complete and matches implementation
- Architecture diagram and ERD match the implementation
- Git branches and incremental history satisfy the specification
- AI Usage section complete; deployment or reproducible Docker deployment available
- Six-minute demo video prepared per the required structure
- Final requirement-to-evidence audit (Section 27) is complete

29. Final Submission Checklist

- Public/evaluator-accessible repo, working MVP (frontend + backend + database)
- Docker setup, .env.example, no secrets committed
- Migrations + seed data using the story cast
- Architecture diagram and ERD kept in sync with the implementation
- master / pre-release / release/v1.0.0 branches with real incremental history
- Meaningful tests (Section 19) + self-explanatory README
- Deployment link (free tier) if available, or documented reproducible Docker deploy
- Six-minute video linked in README
- README AI Usage section
- Requirement-to-evidence audit completed (Section 27)
- Bonus: viral-scale reasoning (1M passengers / 100k drivers) if time allows

30. Hard Rules — Never Do These

- Pay for any infrastructure or service for this challenge
- Commit API keys, passwords, tokens, or .env secrets
- Store plaintext passwords
- Submit a single giant 'initial commit' containing the finished system
- Push feature development directly to master, or manufacture fake history after the fact
- Add technologies only to make the architecture diagram look impressive
- Rely on frontend validation or button visibility as the authorization mechanism
- Polish animations while core data integrity is broken
- Hide AI usage, or include code that can't be explained
- Strip the story cast out of seed data/tests/README in favor of generic placeholders
- Declare the project complete without finishing the requirement-to-evidence audit

Final principle: build the smallest system that is fully correct, explainable, testable, auditable, concurrency-safe, authorized, reproducible, and faithful to this document. Correctness and engineering judgment matter more than feature count or architectural complexity.